

SİSTEM MEMARLIĞI

Arxitekturanın İzahı

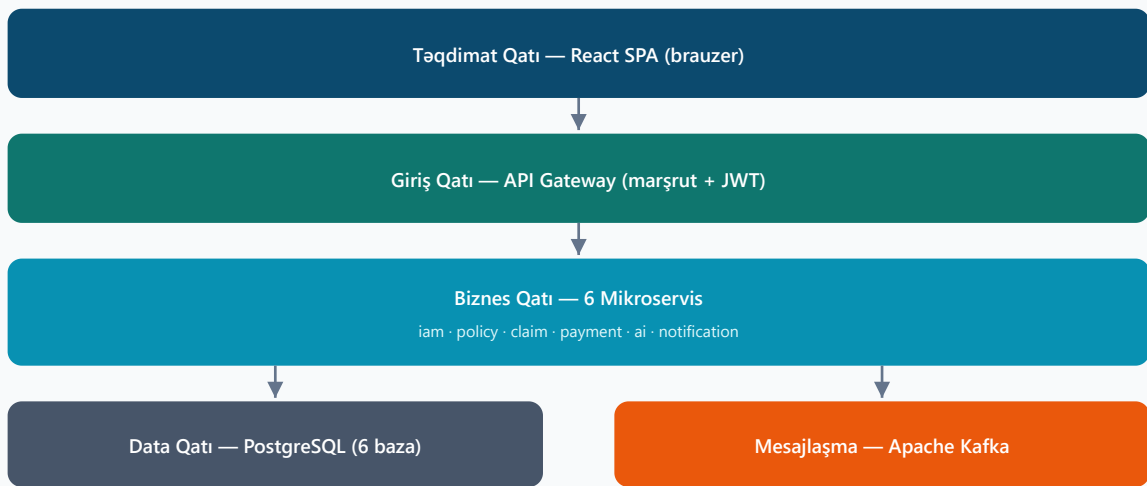
Memarlıq prinsipləri, texnologiya yığını, əlaqə nümunələri,
təhlükəsizlik və texniki borc

E-Sağlamlıq Sığortası — Event-Driven Mikroservis Memarlığı
2026

Memarlıq üslubu

Sistem **event-driven mikroservis** memarlığıdır. Əsas xüsusiyyətlər:

- **Multi-repo:** hər servis öz git repositoriyasıdır — monorepo və ya Gradle `include()` yoxdur. Hər biri öz `settings.gradle`, `build.gradle` və wrapper-i ilə müstəqildir.
- **Database-per-service:** hər servisin öz PostgreSQL bazası var. Servislər bir-birinin bazasına birbaşa müraciət etmir.
- **İki əlaqə kanalı:** sinkron (Gateway üzərindən REST) komandalar üçün, asinxron (Kafka) hadisələrin yayılması üçün.
- **Paylaşılan kontrakt kitabxanası:** `infra` jar bütün enum, event və exception tiplərini mərkəzləşdirir.



Şəkil — Qatlı baxış: təqdimat → giriş → biznes → data/mesajlaşma.

Texnologiya yığını

Sahə	Texnologiya
Dil / Runtime	Java 17
Framework	Spring Boot 3.2.5
Build	Gradle (Groovy DSL + Version Catalog), per-repo
Baza	PostgreSQL 16 (database-per-service)
Mesajlaşma	Apache Kafka (Confluent 7.5.0)
Autentifikasiya	JWT (jjwt 0.12.5)
Mapping	MapStruct 1.5.5
Gateway	Spring Cloud Gateway (WebFlux)
Konteyner	Docker Compose
AI	OpenRouter (OpenAI-uyğun API), google/gemini-2.5-flash, WebClient
Frontend	React + Vite + React Router (+ Vitest/MSW test)

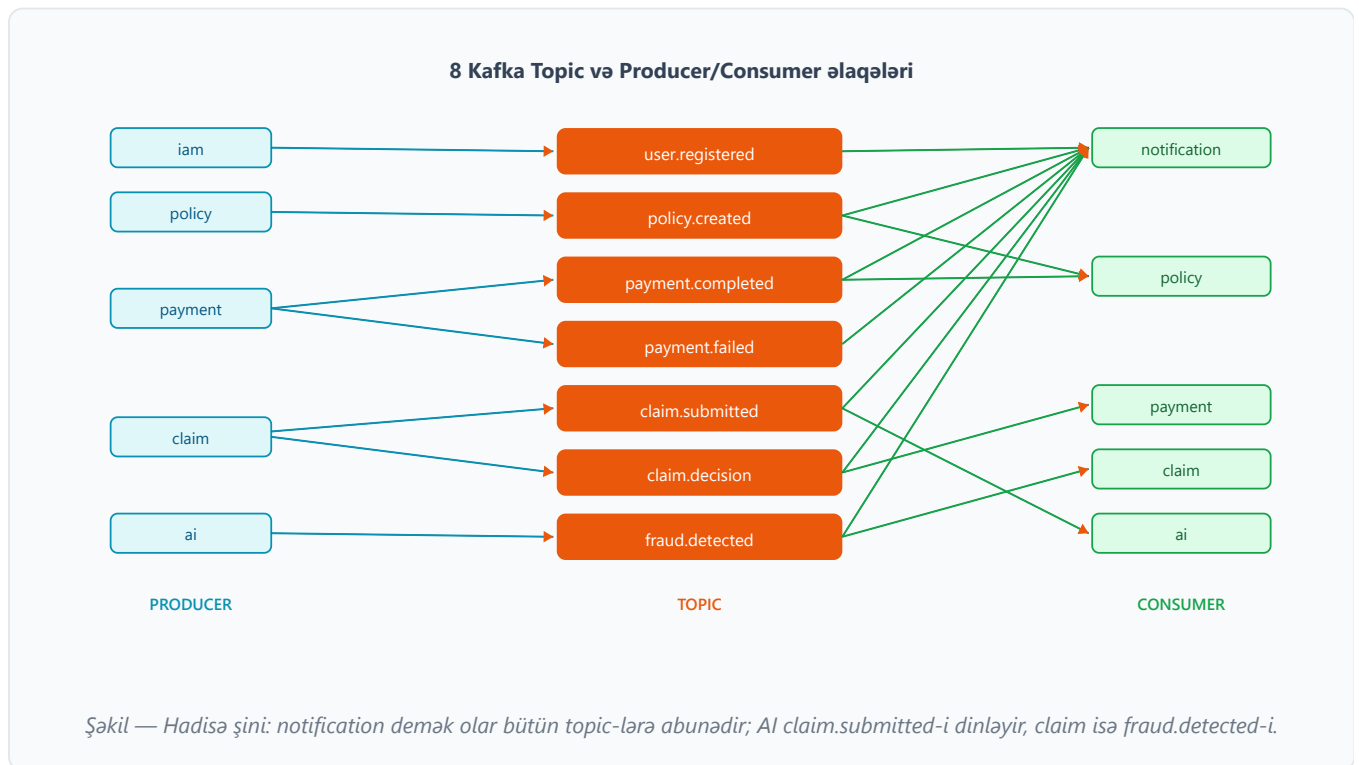
Əlaqə nümunələri

Sinxron — REST (komandalar/sorğular)

İstifadəçi əməlləri (login, plan al, iddia göndər) Gateway üzərindən HTTP ilə gəlir. Gateway JWT-ni doğrulayır, sorğunu uyğun servise marşrutlayır. Hər servis ayrıca yenidən JWT doğrulaması da edir (per-service validation).

Asinxron — Kafka (hadisələr)

Bir servisdə baş verən vacib hadisə (police yarandı, ödəniş tamamlandı) Kafka topic-inə yayımlanır. Maraqlanan servislər abunədir və müstəqil reaksiya verir. Bu, servisləri **boş-bağlı (loosely coupled)** saxlayır.



Servis daxili struktur (təkrarlanan şablon)

Hər biznes servisi eyni paket konvensiyasını izləyir. Bu ardıcılıq komandanın hər servisdə eyni yeri tapmasını asanlaşdırır:

Paket	Məsuliyyət
controller/	REST endpoint-lər, <code>@PreAuthorize</code> rol yoxlaması
service/ + service/impl/	Biznes məntiqi — interfeys + implementasiya
repository/	Spring Data JPA interfeysləri
entity/	JPA entity-ləri (UUID PK, Instant vaxt)
dto/request + dto/response/	Record DTO-lar (giriş/çıxış)
kafka/	XxxEventProducer / XxxEventConsumer
mapper/	MapStruct Entity ↔ DTO çevirmələri
config/ · security/	JWT, SecurityConfig, ConfigurationProperties
exception/	GlobalExceptionHandler + XxxErrorEnum

Kod prinsipləri: Lombok hər yerdə · konstruktor injection (heç vaxt `@Autowired` sahə) · DTO-lar immutable record · bütün cavablar `ApiResponse<T>` zərfində · endpoint-lər `/api/v1/` prefiksli.

Təhlükəsizlik memarlığı

- **JWT axını:** Yalnız IAM token verir; bütün servislər ortaq `jwt.secret` ilə token-i doğrulayır.
- **Subyekt fərqi:** IAM-da JWT subyekti email-dir; digər servislərdə userId (UUID) — çünki onlar istifadəçini userId ilə tanıyır.
- **Rol modeli:** ADMIN / STAFF / CUSTOMER — metod səviyyəsində `@PreAuthorize` .
- **İki qatlı doğrulama:** həm gateway, həm də hər servis JWT-ni yoxlayır (cari model: per-service validation).
- **Sızma qarşısı:** başqasının resursuna müraciət 403 yox, 404 qaytarır.

Məlum problemlər və texniki borc

Memarlıq MVP üçün sağlamdır, lakin prod-a keçməzdən əvvəl həll olunmalı bəzi məsələlər var (tam siyahı `check.md` -də). Önem: ● kritik · ● yüksək · ● orta · ● aşağı.

Önem	Problem	Təsir / Həll
●	JWT secret və DB kreditləri commit-də	Prod-da real secret-ləri env/secret manager ilə inject et
●	Kafka xəta idarəsi / DLT yoxdur	Zədəli payload partition-u kilidləyə bilər; ErrorHandlingDeserializer + DLT əlavə et
●	İkiqat JWT doğrulama; gateway X-User-* başlıqları ölü kod	Bir model seç: gateway-authoritative və ya per-service
●	Bütün servis portları host-a açıq	Prod-da yalnız 8080 + frontend açıq qalsın
●	Yaz+yayımla tranzaksiyalı deyil (outbox yox)	<code>@Transactional</code> (ucuz) və ya transactional outbox (düzgün)
●	notification hələ <code>ddl-auto: update</code>	Flyway/Liquibase migrasiyalarına keç
●	Səhifələmə həddi yoxdur (<code>?size=100000</code>)	<code>@PageableDefault</code> + maksimum həd
●	Police EXPIRED heç vaxt təyin olunmur	Planlaşdırılmış "endDate keçdi" işi əlavə et

Testləşmə memarlığı

Hər servis testlə təchiz olunub (cəmi ~190+ test, hamısı yaşıl). Yığın:

- **Unit:** JUnit 5 + Mockito + AssertJ — service qatı (qərar həddləri, sahiblik/404 məntiqi).
- **İnteqrasiya:** `@SpringBootTest` + `@EmbeddedKafka` + Compose PostgreSQL — hadisə zəncirləri (giriş hadisəsi yayımla, həm DB vəziyyətini, həm çıxış hadisəsini yoxla).
- **Web qatı:** `@WebMvcTest` — rol əsaslı icazələr.
- **AI:** MockWebServer ilə OpenRouter çağırışları.
- **Frontend:** Vitest + React Testing Library + MSW.

Niyə inteqrasiya testləri vacibdir: Məhz onlar ödəniş ikiqat-ödəniş və police aktivləşmə (LazyInitialization) kimi reqressiyaları tutdu.